Question 1

(a) simulator_a.py- Simulates one step of the environment.

Input:

Grid size N

Predator location

Prey location

Predator action

Output:

Next predator location

Next prey location

Reward

Working

1. Predator moves according to action
2. If action is invalid → stays in place
3. Check if predator catches prey
4. If caught:
    - Reward = 1
    - Prey respawns randomly
5. Otherwise:
    - Reward = 0
    - Prey moves randomly to valid neighbors or stays

(b) kernel_b.py- Constructs the exact state-transition kernel.

Size: $|S| \times |A| \times |S|$

Uses full enumeration of all states and actions

Working

For each (s, a):

- Compute next predator location
- If capture:
    - Prey respawns uniformly among $N^2 - 1$cells
- Else:
    - Prey moves uniformly among valid moves

(c) reward_c.py- Defines the reward function:

Size: $|S| \times |A|$

If predator catches the prey- Reward=1, else reward is 0.

(d) policy_d.py- Compute Manhattan distance:

$$d = |x_p - x_r| + |y_p - y_r|$$

Find actions that minimize distance
Assign:

- 50% probability to best actions
- Remaining 50% to other valid actions

(e) induced_kernel_e.py- Computes policy-induced transition kernel:

$$P^{\pi}(s, s') = \sum_{a} \pi(a \mid s) P((s, a), s')$$

(f) induced_reward_f.py- Computes expected reward under policy:

$$R^{\pi}(s) = \sum_{a} \pi(a \mid s) R(s, a)$$

(g) state_value_g.py- Evaluates value function using iterative Bellman updates:

$$V(s) = R^{\pi}(s) + \gamma \sum_{s'} P^{\pi}(s, s') V(s')$$

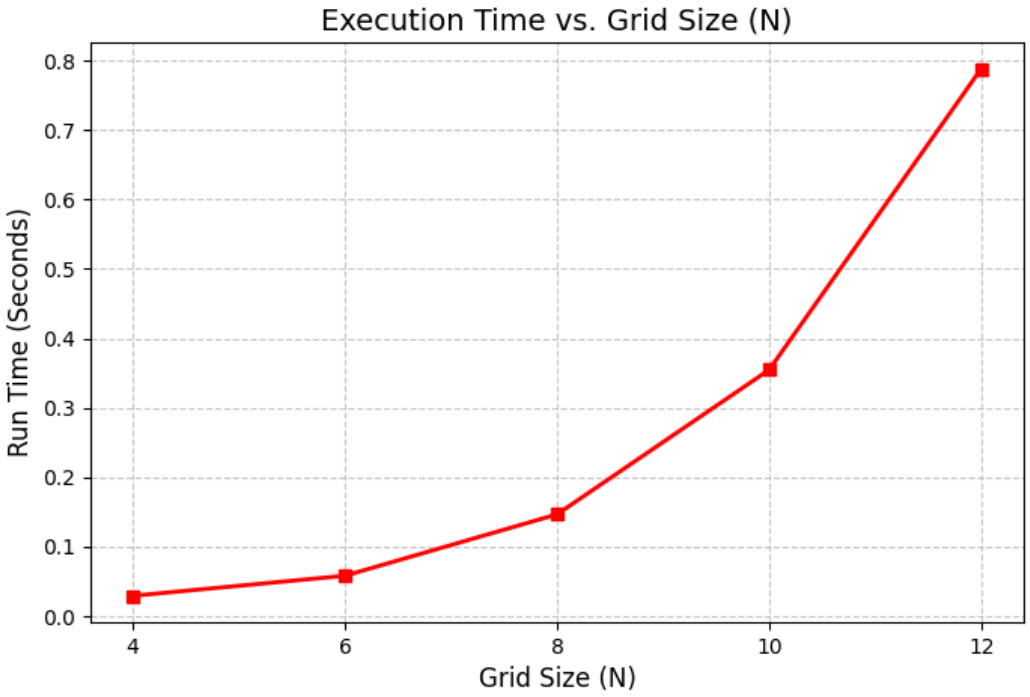
- Discount factor: $\gamma = 0.99$

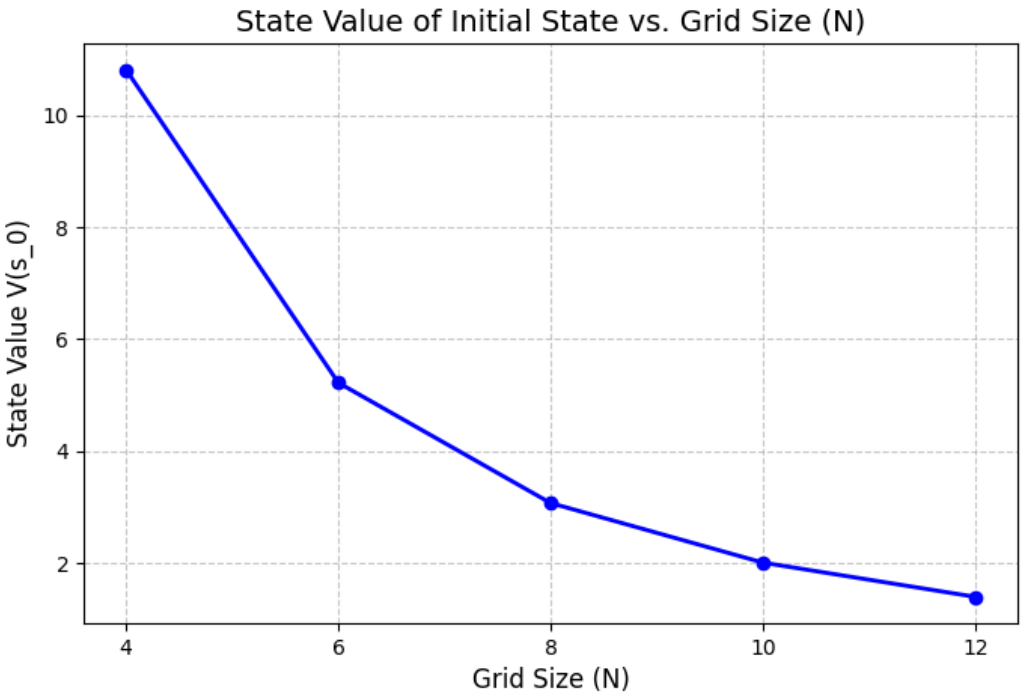
(h) q_value_h.py- Computes Q-function:

$$Q(s, a) = R(s, a) + \gamma \sum_{s'} P((s, a), s') V(s')$$

(j) main_j.py- Runs experiments for different values of N: [4, 6, 8, 10, 12]

Computes value of initial state $((1,1), (N, N))$
Measures runtime
Generates plots:
State value vs N
Runtime vs N





We can clearly observe that computational complexity increases with increase in state space size.